Arrays, Hashing and Two Pointers

Topic 1 of 10

15 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Robot Status Message Deduplication Window

EASY | asked 2026-09-10 | hashmap, simulation

Process timestamped robot messages and display a message only when it has not been displayed within the prior 10 seconds. Hidden arrivals do not refresh the last-displayed timestamp, and the exact 10-second boundary must be clarified. Process a stream of (time, message) status events from a robot. If the same message was displayed within the past 10 seconds, hide the new event. Otherwise display it and update that message's last-displayed time.

```
class Dedup:

    def __init__(self, window=10, boundary_allo
      wed=True):


    def too_soon(self, gap):


    def show(self, t, msg):
```

2. Connected Crop Arrangement on a Grid

MEDIUM | asked 2026-10-04 | grid, array, greedy

Given an N x M grid and K crop counts summing to N*M, fill every cell so each crop's cells form one 4-connected region; return any valid arrangement. A garden is an N x M grid. There are K crop types labeled 1..K; crop i must occupy exactly count[i] cells. Guaranteed: sum(count) = N * M, so every cell is used. For every crop type, all of its cells must form one connected component under 4-directional adjacency (up, down, left, right; diagonals do not count).

```
def arrange_crops(n, m, counts):
```

3. Range Duplicate Check with Constant-Time Queries

MEDIUM | asked 2026-09-29 | array, sliding-window, hashmap, prefix-sum

Decide whether the elements in a range of an array are all unique. Progression: set-based duplicate check -> sliding window -> many range queries where reads dominate, requiring O(1) per query after linear-time preprocessing / updates with linear space. Base: given an array and a range [l, r], determine whether the elements in that range are all unique. The problem is staged: Easy: duplicate detection with a set. Sliding-window duplicate checking.

```
class RangeUnique:

    def __init__(self, a):
```

```
    def rebuild(self):
```

```
    def unique(self, l, r):
```

```
    def update(self, i, value):
```

4. Temperature Monitoring Data Structure

MEDIUM | asked 2026-09-10 | data-structure, streaming, sliding-window

Implement temperature insertion, a moving average, and a constant-time maximum query for a recent window. Clarify whether the window is based on reading count or elapsed days.

```
class Monitor:

    def __init__(self, n):
```

```
    def insert(self, x):
```

5. Longest Same-Color Grid Sequence

MEDIUM | asked 2026-09-08 | array, grid

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line. Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally. Examples: R R R G R R B B G R R R B G B R Answer: 4

```
def longest_same_color_line(g):
```

6. Next-Word Predictor

MEDIUM | asked 2026-09-02 | string, hashmap, data-structure

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.

```
class Predictor:

    def __init__(self, orders=(2, 1)):

    def train(self, sentence):

    def update(self, k, ctx, nxt):

    def predict(self, *context):

    def top_k(self, word, k):
```

7. Piano Hand Movement Grouping

MEDIUM | asked 2026-07-14 | array, simulation, sliding-window

Examples: With notes = [1, 3, 5, 7, 9, 11] and k = 4, the stated output is 2. Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] -> 1, without stating the reach parameter.

```
def groups(notes, k):
```

8. Website Activity Analytics: Unique Users + Session Time

MEDIUM | asked 2026-06-26 | python, hashmap, data-analysis, sql

Given website action records with user_id, timestamp, and activity, compute per-activity unique-user counts and average time spent on the site per user in seconds. The screen paired this Python task with a lighter SQL section and emphasized edge-case discussion over perfect final code. Input is a collection of user-activity records.

```
def unique_users_by_activity(records):
```

9. Gold Chain Split into Two Equal Halves

MEDIUM | asked 2026-04-21 | array, hashmap

Onsite coding interview 1: given a gold chain represented by an array of segment weights, remove exactly one segment and check whether the remaining segments -- when reconnected -- can be cut once more to give two halves of equal total weight.

```
def gold_chain(a):
```

10. Sum of Good Subarrays (Adjacent Diff +/-1)

MEDIUM | asked 2026-02-08 | array, two-pointer

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic +/-1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N). Input: integer array arr of length N. A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1). Singleton subarrays are always good.

```
def good_subarray_sum(a):
```

11. Run-Length Vector Storage + Dot Product

MEDIUM | asked 2026-01-21 | data-structure, two-pointer

Examples: Logical A = [3, 3, 3, 0, 0, 0, 4, 4] Logical B = [1, 1, 1, 1, 2, 2, 2, 2] Encoded A (RLE): [(3,3),(0,3),(4,2)] Encoded B (RLE): [(1,4),(2,4)] Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 + 0 + 16 = 25

```
def rle_dot(a, b):
```

12. Two-Day Log Intersection under Memory Limit

MEDIUM | asked 2025-11-27 | hashmap, two-pointer, streaming

PhD intern coding round 1: two log files with (timestamp, obj_id, client_id) entries. Find obj_ids that (a) appear in both files and (b) have at least two distinct client_ids. Input: two log files; each line is (timestamp, obj_id, client_id). Find every obj_id such that: (a) appears in both files, and (b) is associated with at least 2 distinct client_ids (aggregated across both files).

```
def log_objects(day1, day2):
```

13. Rotate Any mxn 2D Matrix

MEDIUM | asked n/a | array, grid

Examples: [[1,2,3],[4,5,6]] (2x3) rotated 90deg CW -> [[4,1],[5,2],[6,3]] (3x2).

```
def rotate_cw(matrix):
```

14. Maximum-Area Rectangle from Planar Points

HARD | asked 2026-09-29 | math, hashmap

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Input: a list of 2D coordinate points. Return the maximum area of any rectangle whose four vertices are all input points. Rectangle sides do not need to be parallel to the coordinate axes.

```
def max_rect_area(points):
```

15. Necklace Cut into Two Halves with Equal D/R Counts

HARD | asked 2026-05-03 | string, two-pointer, math-reasoning

Examples: s = "DRDR" -> cut between index 1 and 2: "DR" + "DR". s = "DDRR" -> 1-cut not balanced ("DD" vs "RR"); rotate, two cuts at offsets 1 and 3: pieces "DR" and "DR".

```
def necklace_two_cuts(s):
```